



UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI
INGEGNERIA ELETTRICA ELETTRONICA ED INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

RELAZIONE TECNICA

HW3 - MICROSERVIZI PER GESTIONE UTENTI E MONITORAGGIO VOLI
AEREI

Alunni:

Arena Giorgia
Tornabene Alessio

Docenti:

Di Stefano Antonella
Morana Giovanni

PhD:

Genovese Alessandro

INDICE

- 1. INTRODUZIONE E OBIETTIVI3**
 - 1.1 EVOLUZIONE DEL SISTEMA: DA DOCKER COMPOSE A KUBERNETES3*
 - 1.2 OBIETTIVI DEL WHITE-BOX MONITORING4*
- 2. ARCHITETTURA CLOUD-NATIVE SU KUBERNETES6**
 - 2.1 SCELTA DELL'INFRASTRUTTURA: CLUSTER KIND E STANDARD CLOUD NATIVE COMPUTING FOUNDATION7*
 - 2.1.1 TOPOLOGIA DEL CLUSTER: ARCHITETTURA SINGLE-NODE (CONTROL PLANE & WORKER)7*
 - 2.2 GESTIONE DELLO STATO: SEPARAZIONE SEMANTICA (STATEFULSET VS DEPLOYMENT)8*
 - 2.3 STORAGE PERSISTENTE E DISACCOPPIAMENTO (PVC)9*
 - 2.4 NETWORKING E SERVICE DISCOVERY AVANZATA9*
- 3. IL SISTEMA DI MONITORAGGIO (PROMETHEUS)..... 11**
 - 3.1 ARCHITETTURA PULL-BASED E KUBERNETES SERVICE DISCOVERY 11*
 - 3.2 STRUMENTAZIONE DEL CODICE (WHITE-BOX MONITORING) 11*
 - 3.3 ANALISI DELLE METRICHE IMPLEMENTATE 12*
 - 3.4 VALIDAZIONE TRAMITE PROMQL..... 13*
- 4. OTTIMIZZAZIONE E GESTIONE RISORSE..... 15**
 - 4.1 RESOURCE QUOTAS & LIMITS (QUALITY OF SERVICE - QoS)..... 15*
 - 4.2 GESTIONE AVANZATA DELLO STORAGE KAFKA (SIZE-BASED RETENTION)..... 15*
- 5. ANALISI DELLA RESILIENZA E SCENARI DI GUASTO 17**
 - 5.1 OSSERVABILITÀ DEI GUASTI (WHITE-BOX) 17*
 - 5.2 PERSISTENZA DEI DATI E SELF-HEALING..... 17*
- 6. ISTRUZIONI DI DEPLOYMENT E TESTING..... 19**
 - 6.1 COMANDI ESSENZIALI PER FAR PARTIRE IL PROGETTO 19*
 - 6.2 COMANDI PER IL TESTING DEL PROGETTO 21*
 - 6.2 VALIDAZIONE EMPIRICA TRAMITE QUERY PROMQL 23*
- 7. CONCLUSIONI 28**

1. INTRODUZIONE E OBIETTIVI

1.1 Evoluzione del Sistema: Da Docker Compose a Kubernetes

Il presente progetto rappresenta un salto qualitativo rispetto all'architettura a microservizi sviluppata nell'Homework 2. Mentre la precedente iterazione si concentrava sulla definizione funzionale dei servizi e dei protocolli di comunicazione, questo elaborato sposta il focus sull'**orchestrazione dichiarativa**, sulla **scalabilità orizzontale** e sulla **resilienza infrastrutturale**.

La migrazione da un ambiente Docker Compose (imperativo e locale) a un cluster **Kubernetes** (dichiarativo e distribuito, simulato tramite **Kind**) ha comportato una ridefinizione profonda dei componenti:

- **Gestione Avanzata dello Stato (Stateful vs Stateless):**

Abbiamo superato il concetto generico di "container" introducendo una distinzione architetturale netta.

- I microservizi applicativi (*User Manager, Data Collector, Alert System*) sono stati distribuiti come **Deployments**, trattandoli come entità effimere ("Cattle") scalabili e sostituibili senza perdita di dati.
- I componenti critici per la persistenza (*PostgreSQL, MongoDB, Kafka*) sono stati modellati come **StatefulSets**. Questa primitiva Kubernetes garantisce identità di rete stabili, storage persistente ordinato e una gestione controllata del ciclo di vita, fondamentale per sistemi distribuiti che necessitano di consenso e replicazione.

- **Persistenza e Disaccoppiamento dello Storage:**

L'introduzione dei **Persistent Volume Claims (PVC)** ha permesso di disaccoppiare il ciclo di vita del dato da quello del processo. A differenza dei volumi Docker standard, i PVC garantiscono che, in caso di crash o riavvio di un Pod (es. il broker Kafka), il nuovo processo si riagganci automaticamente allo stesso disco virtuale, azzerando il rischio di data loss.

- **Segregazione e QoS (Quality of Service):**

Attraverso la definizione di Requests e Limits per CPU e Memoria nei manifest YAML, il sistema implementa un controllo granulare delle risorse. Questo previene che un singolo microservizio compromesso (es. memory leak) possa saturare l'intero nodo, garantendo l'isolamento dei guasti.

1.2 Obiettivi del White-Box Monitoring

In accordo con le specifiche di progetto, è stato introdotto un layer di monitoraggio **White-Box** basato su **Prometheus**, integrato nativamente nel cluster tramite meccanismi di **Service Discovery**.

A differenza di un approccio Black-Box (che si limiterebbe a verificare la raggiungibilità HTTP o lo stato "up/down" dei container dall'esterno), l'approccio White-Box adottato prevede la **strumentazione diretta del codice applicativo**. Abbiamo modificato la logica dei microservizi Python (*User Manager*, *Data Collector* e *Notifier*) iniettando sonde software (tramite la libreria *prometheus_client*) capaci di esporre metriche interne di business e di performance.

Questo ci permette una diagnosi proattiva e granulare su tre livelli:

1. **Performance del Database (Query Observability):**

Non ci limitiamo a sapere se il DB risponde, ma misuriamo *come* risponde. Tramite metriche di tipo **GAUGE** (es. *user_db_last_query_seconds*, *mongo_last_query_seconds*), monitoriamo in tempo reale la latenza delle singole operazioni (INSERT vs SELECT), permettendo di identificare colli di bottiglia specifici nelle query SQL o nelle aggregazioni NoSQL.

2. **Affidabilità delle Dipendenze Esterne:**

Nel *Data Collector*, abbiamo strumentato il **Circuit Breaker** verso l'API OpenSky. Tramite metriche **COUNTER** etichettate (es. *opensky_requests_total* con *label status="circuit_open"*), possiamo visualizzare su Prometheus la frequenza di apertura del circuito, correlando i fallimenti della rete esterna con il comportamento del sistema interno.

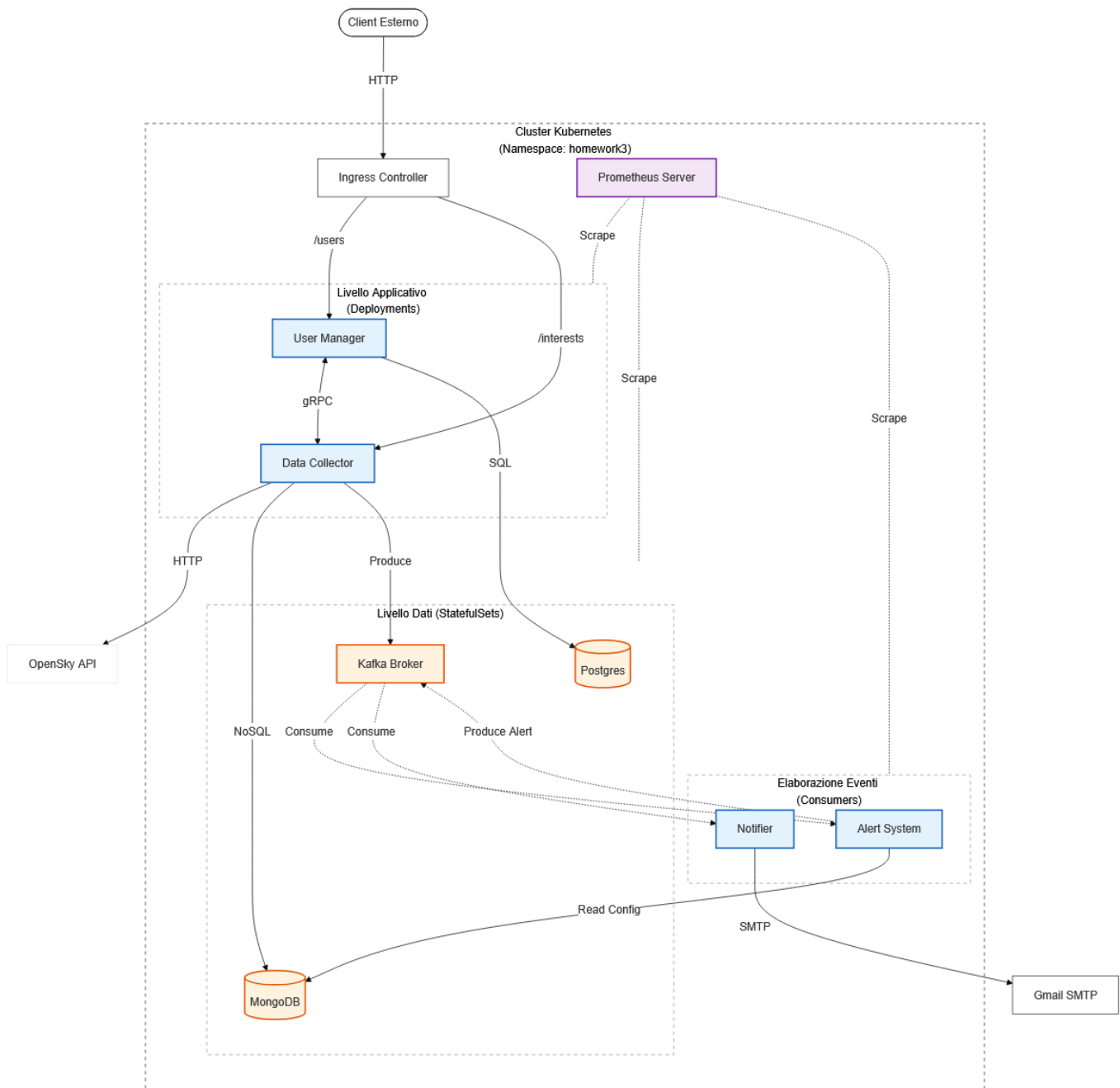
3. **Throughput e Business Logic:**

Monitoriamo il carico reale del sistema (richieste al secondo, utenti registrati, voli processati) utilizzando contatori cumulativi che persistono anche in caso di riavvio del processo di raccoglimento dei dati, grazie alla natura stateful del server Prometheus.

In sintesi, il sistema di monitoraggio non è un componente passivo aggiunto a posteriori, ma parte integrante dell'architettura software, progettato per offrire visibilità profonda sullo stato di salute dell'intero stack distribuito.

2. ARCHITETTURA CLOUD-NATIVE SU KUBERNETES

L'architettura del sistema è stata completamente ridisegnata sfruttando le primitive native di Kubernetes per garantire la robustezza, la scalabilità e la resilienza tipiche dei sistemi di produzione Enterprise. La transizione da un approccio imperativo (Docker Compose) a uno dichiarativo (Kubernetes Manifests) ha permesso di definire lo stato desiderato dell'infrastruttura, delegando al Control Plane di Kubernetes il compito di mantenerlo.



2.1 Scelta dell'Infrastruttura: Cluster Kind e Standard Cloud Native Computing Foundation

Per simulare fedelmente un ambiente Cloud-Native su risorse locali, è stato adottato **Kind (Kubernetes IN Docker)**. A differenza di soluzioni come Minikube che spesso richiedono l'uso di VM pesanti, Kind utilizza container Docker per simulare i nodi del cluster, riducendo l'overhead sulle risorse host.

Questa scelta garantisce che i manifest YAML prodotti siano pienamente conformi agli standard della **Cloud Native Computing Foundation**. Ciò assicura la portabilità: la stessa configurazione testata localmente può essere deployata su provider cloud gestiti come AWS EKS, Google GKE o Azure AKS senza modifiche sostanziali.

2.1.1 Topologia del Cluster: Architettura Single-Node (Control Plane & Worker)

Per ottimizzare l'utilizzo delle risorse sulla macchina locale, abbiamo definito una topologia di cluster personalizzata tramite il file `kind-config.yaml`. In un ambiente di produzione Kubernetes standard, i nodi di **Control Plane** (che gestiscono l'orchestrazione) e i nodi **Worker** (che eseguono i carichi di lavoro) sono separati fisicamente per garantire isolamento e sicurezza.

Tuttavia, per rispettare i vincoli del progetto e minimizzare l'overhead, abbiamo configurato un **cluster a nodo singolo** che ricopre un ruolo ibrido:

1. **Ruolo Control Plane:** Gestisce l'API Server, lo Scheduler e l'etcd (definito con `role: control-plane`).
2. **Ruolo Worker:** Grazie alla configurazione di Kind, questo nodo è abilitato ad accettare lo scheduling dei Pod applicativi (User Manager, Kafka, Database), comportandosi effettivamente come un nodo di lavoro.

Inoltre, la configurazione `kind-config.yaml` è stata arricchita con due direttive fondamentali per l'esposizione dei servizi:

- **Kubeadm Config Patches:** Abbiamo iniettato l'etichetta `node-labels: "ingress-ready=true"`. Questo permette all'Ingress Controller NGINX di identificare specificamente questo nodo per il routing del traffico HTTP.
- **Port Mapping (Network Bridge):** Abbiamo configurato il `extraPortMappings` per creare un tunnel diretto tra le porte del container Docker (il nodo) e le porte della macchina Host:

- **Porte 80/443:** Mappate per permettere l'accesso ai microservizi tramite l'Ingress (es. chiamate API HTTP).
- **Porta 9090:** Mappata sulla NodePort 30090 per permettere l'accesso diretto alla dashboard di Prometheus (localhost:9090) senza dover utilizzare comandi di `kubectl port-forward`.

Questa configurazione ci ha permesso di simulare un'intera infrastruttura Cloud (Rete, Storage, Compute) all'interno di un singolo container Docker, mantenendo però la completa compatibilità con le API standard di Kubernetes.

2.2 Gestione dello Stato: Separazione Semantica (*StatefulSet* vs *Deployment*)

Una delle decisioni architetturali più critiche è stata la netta separazione tra carichi di lavoro *Stateless* e *Stateful*, mappandoli sugli oggetti Kubernetes più idonei:

- **Microservizi Stateless (*Deployment*):**

I servizi applicativi (*User Manager*, *Data Collector*, *Alert System*, *Notifier*) sono gestiti tramite l'oggetto **Deployment**.

- **Logica:** Questi servizi contengono esclusivamente la business logic e non mantengono stato locale persistente.
- **Vantaggio Architettuale:** Kubernetes tratta questi Pod come entità effimere ("Cattle"). Il cluster può riavviarli, scalarli orizzontalmente (aumentando il numero di repliche) o spostarli su nodi diversi in caso di guasto hardware, senza alcuna perdita di dati e senza interrompere il servizio, garantendo l'High Availability.

- **Componenti Stateful (*StatefulSet*):**

Per i livelli di persistenza e messaggistica (*PostgreSQL*, *MongoDB*, *Kafka*, *Prometheus*), l'uso dei *Deployment* non era idoneo poiché questi sistemi richiedono un'identità stabile. È stato scelto l'oggetto **StatefulSet** che garantisce:

- **Identità di Rete Stabile (*Sticky Identity*):** I pod ottengono nomi hostname sequenziali e deterministici (es. *kafka-0*, *mongo-0*) invece di hash casuali. Questo è critico per sistemi distribuiti come Kafka, dove i broker devono essere identificabili univocamente per gestire la replicazione delle partizioni e l'elezione del leader.

- o **Storage Stable:** Assicura che, se un pod viene riavviato, venga ricollegato allo stesso volume persistente che aveva in precedenza.
- o **Ordine di Avvio Sequenziale:** Garantisce un'inizializzazione ordinata, evitando race condition durante il bootstrap del cluster (es. il nodo master del DB parte prima delle repliche).

2.3 Storage Persistente e Disaccoppiamento (PVC)

Per risolvere il problema della volatilità dei container, è stato implementato un layer di storage persistente basato su **Persistent Volume Claims (PVC)**.

Ogni StatefulSet richiede dinamicamente uno spazio su disco (configurato a 250Mi per componente nei nostri *manifest.yaml*) tramite lo StorageClass standard.

- **Design Pattern:** Il Pod (capacità di calcolo) è completamente disaccoppiato dal Volume (dati).
 - o *Scenario di resilienza:* Se il pod postgres-0 crasha a causa di un errore software, Kubernetes ne avvia automaticamente uno nuovo. Grazie al binding del PVC, il nuovo pod monta istantaneamente il volume precedente (postgres-storage-postgres-0), trovando il database intatto. Questo garantisce **Zero Data Loss** anche in caso di fallimenti critici dei processi.

2.4 Networking e Service Discovery Avanzata

La comunicazione interna non avviene più tramite i link statici di Docker legacy, ma attraverso l'astrazione dei **Service Kubernetes**, sfruttando il DNS interno del cluster:

- **ClusterIP (per i Microservizi):**

Utilizzato per *User Manager*, *Data Collector* e gli altri servizi stateless. Fornisce un indirizzo IP virtuale stabile che bilancia il traffico (Load Balancing interno) verso i pod disponibili. I servizi si invocano semplicemente tramite nome DNS, rendendo il codice agnostico rispetto alla topologia di rete.

- **Headless Service (per StatefulSet):**

Per i servizi *StatefulSet*, è stato configurato un servizio di tipo **Headless** (impostando clusterIP: None nel manifest YAML).

- o **Motivazione Tecnica:** Questo permette la risoluzione DNS diretta dei singoli IP dei Pod (es. *kafka-0.kafka-service.homework3.svc.cluster.local*).
- o **Implementazione nel Codice:** Questa feature è stata fondamentale per risolvere i problemi di connessione del client Kafka. Nel file *deployments.yaml*, la variabile d'ambiente `KAFKA_BOOTSTRAP_SERVERS` è stata configurata per puntare esplicitamente all'identità di rete stabile del broker (*kafka-0.kafka-service...:9092*), permettendo ai produttori e consumatori di negoziare correttamente la connessione e la gestione dei metadati delle partizioni.

3. IL SISTEMA DI MONITORAGGIO (PROMETHEUS)

In conformità con i requisiti del progetto (monitoraggio *White-Box*), il sistema è stato esteso integrando **Prometheus** come componente infrastrutturale di prima classe all'interno del cluster Kubernetes. L'obiettivo non è limitarsi a verificare se i container sono attivi (liveness), ma analizzare *come* stanno performando internamente, correlando metriche applicative e infrastrutturali.

3.1 Architettura Pull-Based e Kubernetes Service Discovery

L'architettura di monitoraggio adotta il modello **Pull**: non sono i microservizi a inviare dati (Push), ma è Prometheus che interroga periodicamente (scraping) gli endpoint HTTP /metrics esposti dai servizi.

In un ambiente dinamico come Kubernetes, dove i Pod sono effimeri e i loro indirizzi IP cambiano a ogni riavvio, la configurazione statica dei target è impraticabile. Per risolvere questo problema, abbiamo configurato la **Kubernetes Service Discovery** nel file prometheus.yaml.

- **Meccanismo di Discovery:** Prometheus è configurato con role: pod, istruendo il server a interrogare direttamente le API del Kubernetes Control Plane per ottenere la lista aggiornata in tempo reale di tutti i Pod attivi nel namespace.
- **Filtraggio Dinamico (Relabeling):** Poiché non tutti i pod necessitano di monitoraggio, abbiamo implementato regole di relabel_configs che filtrano i target basandosi sulle annotazioni definite nel deployments.yaml. Il server monitora esclusivamente i Pod che espongono:

annotations:

prometheus.io/scrape: "true"

prometheus.io/port: "8000"

Grazie a una regola di action: replace configurata in Prometheus, la porta specificata nell'annotazione sovrascrive quella di default, permettendo a ogni microservizio di esporre le metriche su porte dedicate (es. User Manager su 8000, Notifier su 8002) senza conflitti.

Questa configurazione garantisce la scalabilità automatica: se un Deployment scala da 1 a 5 repliche, Prometheus scoprirà automaticamente i nuovi target senza alcun intervento manuale.

3.2 Strumentazione del Codice (White-Box Monitoring)

L'approccio White-Box ha richiesto la modifica del codice sorgente Python (app.py) dei microservizi, integrando la libreria ufficiale prometheus_client. Questa scelta architetturale ci permette di esporre metriche di business che il livello infrastrutturale ignorerebbe.

Arricchimento delle Metriche (Labeling):

Per rendere le metriche utili in un contesto distribuito, ogni misurazione viene arricchita con etichette (*labels*) contestuali.

- **Identità del Nodo (Downward API):** Per identificare su quale macchina fisica sta girando un pod lento, abbiamo utilizzato la Downward API di Kubernetes. La variabile d'ambiente NODE_NAME viene iniettata nel container e usata come label nella metrica:

```
REQUEST_COUNT = Counter(..., labelnames=['service', 'node', 'endpoint'])
```

In questo modo, possiamo isolare problemi di performance legati a uno specifico nodo del cluster (es. "noisy neighbor").

3.3 Analisi delle Metriche Implementate

Abbiamo definito un set di metriche standardizzate per i microservizi critici, distinguendo tra misurazioni istantanee (Gauge) e cumulative (Counter).

A. User Manager (PostgreSQL - Performance Relazionali)

Questo servizio gestisce dati strutturati e transazionali. Il monitoraggio si concentra sulla latenza delle query SQL.

1. user_db_last_query_seconds (GAUGE)

- o *Semantica:* Rappresenta la durata esatta dell'ultima operazione eseguita sul database.
- o *Utilizzo:* Etichettata per operation_type (SELECT, INSERT, DELETE), permette di identificare istantaneamente colli di bottiglia (es. un lock su una tabella).

2. user_db_total_query_seconds (COUNTER)

- o *Semantica:* Somma cumulativa del tempo speso in operazioni DB. Fondamentale per calcolare la media nel tempo tramite la funzione rate().

B. Data Collector (MongoDB & Integrazioni Esterne)

Servizio I/O bound che interagisce con sistemi esterni. Le metriche riflettono la resilienza.

1. **mongo_last_query_seconds (GAUGE)**

- o *Semantica*: Misura il tempo di risposta del database NoSQL per operazioni batch su documenti JSON.

2. **opensky_requests_total (COUNTER)**

- o *Monitoraggio della Resilienza*: Cruciale per validare il **Circuit Breaker**. La label status traccia:
 - success: Richieste ok.
 - circuit_open: Richieste bloccate preventivamente. Un incremento di questo valore dimostra che il sistema si sta "autoproteggendo" da un fallimento a cascata.

C. Notifier (SMTP & Alerting)

Questo componente gestisce l'invio critico delle notifiche via email.

1. **email_latency (GAUGE)**

- o *Semantica*: Misura il tempo impiegato per l'handshake e l'invio effettivo dell'email al server SMTP (Gmail).
- o *Utilizzo*: Permette di rilevare degradazioni nella rete esterna o nel server SMTP.

2. **emails_total (COUNTER)**

- o *Semantica*: Conta il numero totale di tentativi di invio, etichettati per esito.
- o *Labels*:
 - status="success": Email inviata correttamente.
 - status="auth_error": Problemi di credenziali (Secret Kubernetes errati).
 - status="kafka_error": Problemi di comunicazione con il Broker.

3.4 Validazione tramite PromQL

Per dimostrare l'efficacia del monitoraggio, il sistema permette di eseguire query complesse tramite la dashboard di Prometheus. Esempi significativi includono:

- **Calcolo del Throughput API:**

rate(opensky_requests_total[5m])

- **Identificazione Query Lente (Top K):**

topk(3, user_db_last_query_seconds)

- **Tasso di Errore Notifiche:**

rate(emails_total{status!="success"}[5m])

Questa infrastruttura trasforma i database e i servizi esterni da "scatole nere" a componenti trasparenti, permettendo decisioni operative basate su dati reali.

4. OTTIMIZZAZIONE E GESTIONE RISORSE

Dovendo simulare un ambiente distribuito complesso (8+ container tra database, broker e microservizi) su un cluster locale (Kind) con risorse finite, l'efficienza non è stata un'opzione ma un requisito vincolante. Abbiamo adottato un approccio di "**Resource Constraints Engineering**" per garantire la stabilità del sistema.

4.1 Resource Quotas & Limits (Quality of Service - QoS)

Nel manifesto deployments.yaml, abbiamo abbandonato la configurazione di default (BestEffort) per definire esplicitamente le risorse per ogni container.

- **Requests:** Abbiamo impostato valori di richiesta minimi (es. cpu: "50m", memory: "60Mi").
 - Questo valore garantisce al pod la quantità minima di risorse per *avviarsi*. Mantenendo le requests basse, lo scheduler di Kubernetes può inserire molti pod sullo stesso nodo virtuale, massimizzando la densità di deployment senza sprechi.
- **Limits :** Abbiamo impostato limiti superiori rigidi (es. memory: "250Mi").
 - Se un microservizio (es. il *Data Collector* che processa grandi JSON) dovesse subire un memory leak o un picco anomalo, il kernel Linux interverrebbe uccidendo il processo.
 - *Vantaggio:* Questo sacrificio controllato del singolo pod protegge l'intero nodo: si evita che un servizio "impazzito" consumi tutta la RAM disponibile, mandando in crash componenti critici come *Kafka* o il *Kubelet* stesso.

4.2 Gestione Avanzata dello Storage Kafka (Size-based Retention)

Una delle criticità maggiori riscontrate durante i test di carico riguardava la gestione dei log di Kafka. In una configurazione standard, Kafka conserva i messaggi per **7 giorni**. Tuttavia, avendo assegnato ai PVC una dimensione contenuta di **250Mi** (per

risparmiare spazio su disco host), il volume si saturava rapidamente a causa del flusso continuo di dati di volo, portando il broker in stato di *CrashLoopBackOff*.

Per risolvere il problema strutturalmente, abbiamo modificato la strategia di retention nel kafka.yaml:

- **Configurazione:** Variabile d'ambiente
KAFKA_LOG_RETENTION_BYTES=100000000 (100MB).
- **Analisi del Capacity Planning:**
 - *Total PVC Size:* 250MB.
 - *Log Retention:* 100MB.
 - *Safety Margin:* 150MB.
- **Risultato Operativo:** Abbiamo trasformato Kafka in un **Buffer Circolare**. Quando i log raggiungono i 100MB, i segmenti più vecchi vengono eliminati automaticamente per fare spazio ai nuovi. Il margine di sicurezza di 150MB garantisce spazio sufficiente per:
 1. I file di indice interni di Kafka.
 2. L'overhead del file system.
 3. Eventuali picchi temporanei prima che il Garbage Collector dei log intervenga.

Questa configurazione garantisce che il servizio rimanga **sempre disponibile** (Availability) anche sotto un flusso di dati infinito, prevenendo fisicamente il riempimento del disco virtuale.

5. ANALISI DELLA RESILIENZA E SCENARI DI GUASTO

La robustezza del sistema è stata validata simulando scenari di guasto critici. Rispetto all'Homework 2, dove la gestione degli errori era confinata alla logica applicativa (codice Python), in questa iterazione abbiamo delegato gran parte della responsabilità di "Self-Healing" all'orchestratore Kubernetes.

5.1 Osservabilità dei Guasti (White-Box)

L'introduzione di Prometheus ha radicalmente trasformato la gestione operativa degli incidenti, passando da un approccio reattivo a uno proattivo.

- **Scenario: Fallimento API OpenSky (Circuit Breaker Monitoring):**
Nell'architettura precedente, l'apertura del Circuit Breaker nel *Data Collector* era visibile solo ispezionando manualmente i container (docker logs). Con l'attuale strumentazione White-Box, abbiamo mappato gli stati del Circuit Breaker sulla metrica Prometheus `opensky_requests_total`.
 - **Rilevamento:** Quando l'API esterna smette di rispondere, il codice Python incrementa il contatore con la label `status="circuit_open"`.
 - **Vantaggio:** Questo ci permette di visualizzare graficamente la frequenza di apertura del circuito. Un picco improvviso di questa metrica indica inequivocabilmente un disservizio esterno, permettendo di distinguere tra errori interni ed esterni, senza dover accedere alla shell del pod.

5.2 Persistenza dei Dati e Self-Healing

Abbiamo verificato la capacità del sistema di sopravvivere al crash dei componenti stateful (*Kafka* e *MongoDB*), dimostrando l'efficacia dell'accoppiata **StatefulSet + PVC**.

- **Scenario Simulato:** Terminazione forzata ("Kill") del processo `kafka-0`.
- **Sequenza di Ripristino Automatica:**

1. **Rilevamento (Liveness Probe):** Il Kubelet rileva che il container non risponde più ai controlli di salute configurati nel manifesto (es. check sulla porta 9092).
 2. **Rescheduling Deterministico:** Il controller dello StatefulSet avvia un nuovo pod sostitutivo. A differenza dei Deployment, che creano pod con nomi casuali (es. app-xg52s), il nuovo pod assume **esattamente lo stesso nome** del precedente: kafka-0.
 3. **Volume Remounting:** Grazie all'identità stabile, Kubernetes ricollega automaticamente il **Persistent Volume Claim** (pvc-kafka-0) al nuovo container.
- **Risultato:** Al riavvio, il broker Kafka ritrova intatti i file di log sul disco. Non c'è perdita di messaggi (semantica *At-Least-Once* preservata) né necessità di riconfigurare i client, poiché l'endpoint DNS kafka-0.kafka-service punta automaticamente al nuovo indirizzo IP del pod rigenerato.

Questa architettura dimostra una resilienza a "doppio strato":

1. **Livello Applicativo:** Retry Policy e Circuit Breaker gestiscono errori transitori.
2. **Livello Infrastrutturale:** Kubernetes gestisce il riavvio dei processi e l'integrità dello storage fisico.

6. ISTRUZIONI DI DEPLOYMENT E TESTING

Per garantire la riproducibilità dell'ambiente e facilitare la valutazione in sede d'esame, è stata definita una procedura operativa rigorosa che copre l'intero ciclo di vita del cluster, dal reset iniziale alla validazione delle metriche.

6.1 Comandi essenziali per far partire il progetto

1. Creare il nuovo cluster partendo dalla configurazione del file kind-config.yaml (spostarsi nella directory k8s) :

kind create cluster --config kind-config.yaml --name homework3

2. Installare il controller nginx per smistare il traffico di rete nelle porte 80 (HTTP) o 443 (HTTPS se si abilitano i certificati SSL o TLS ma nel nostro progetto non era richiesto):

kubectl apply -f <https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml>

3. Nel terminale se si è nella directory k8s tornare in quella precedente con cd.. dove vi è la cartella del progetto e costruiamo le immagini e le carichiamo su kind:

USER MANAGER

- docker build -t user-manager:v1 ./user_manager
- kind load docker-image user-manager:v1 --name homework3

DATA COLLECTOR

- docker build -t data-collector:v1 ./data_collector
- kind load docker-image data-collector:v1 --name homework3

ALERT SYSTEM

- `docker build -t alert-system:v1 ./alert_system`
- `kind load docker-image alert-system:v1 --name homework3`

NOTIFIER

- `docker build -t notifier:v1 ./notifier`
- `kind load docker-image notifier:v1 --name homework3`

4. Torniamo nella cartella k8s e applichiamo i nostri *manifest.yaml*:

1. Namespace e Secrets (Base)

- `kubectl apply -f namespace.yaml`
- `kubectl apply -f secrets.yaml`

2. Database e Kafka (Infrastruttura)

- `kubectl apply -f database.yaml`
- `kubectl apply -f kafka.yaml`

3. I Servizi

- `kubectl apply -f deployments.yaml`

4. Rete e Sicurezza

- `kubectl apply -f network-policy.yaml`
- `kubectl apply -f ingress.yaml`

5. Monitoraggio (Prometheus)

- `kubectl apply -f prometheus.yaml`

6.2 Comandi per il testing del progetto

Verifichiamo che i pod siano vivi (dobbiamo averli in running, i database perdono tempo perché devono allocare i dati su disco)

kubectl get pods -n homework3

oppure per vedere in live come switchano dallo stato creating in running :

kubectl get pods -n homework3 --watch

Comandi utili se vogliamo staccare i container senza eliminare il cluster:

docker stop homework3-control-plane (stop del nodo master)

per riavviarlo :

docker start homework3-control-plane (riavvio del nodo master)

Per vedere i log così verifichiamo se arrivano notifiche o se funzionano i microservizi (avviarli in terminali diversi) :

kubectl logs -l app=user-manager -n homework3 -f

kubectl logs -l app=data-collector -n homework3 -f

kubectl logs -l app=alert-system -n homework3 -f

kubectl logs -l app=notifier -n homework3 -f

Se si vuole eliminare il cluster completo:

kind delete cluster --name homework3

NB.

In Kubernetes, eliminare un Deployment non cancella i dati associati, poiché i

Persistent Volume Claims (PVC) sono progettati per sopravvivere ai Pod. Per effettuare una dimostrazione "Clean Slate" (partendo da zero utenti e zero voli), è necessario invocare una bonifica completa.

Verifichiamo i dati persistenti :

- **kubectl get pvc -n homework3**

Per eliminare i dati persistenti ed evitare che si esaurisce il disco :

Mongo

- **kubectl delete pvc mongo-storage-mongo-0 -n homework3**

Per forzare l'eliminazione dei dati si consiglia di utilizzare il seguente comando:

- **kubectl patch pvc mongo-storage-mongo-0 -n homework3 -p '{"metadata":{"finalizers":null}}'**

Postgres

- **kubectl delete pvc postgres-storage-postgres-0 -n homework3**

Per forzare l'eliminazione dei dati si consiglia di utilizzare il seguente comando:

- **kubectl patch pvc postgres-storage-postgres-0 -n homework3 -p '{"metadata":{"finalizers":null}}'**

Prometheus

- **kubectl delete pvc prometheus-storage-prometheus-server-0 -n homework3**

Per forzare l'eliminazione dei dati si consiglia di utilizzare il seguente comando:

- **Kubectl patch pvc prometheus-storage-prometheus-server-0 -n homework3 -p '{"metadata":{"finalizers":null}}'**

Kafka

- **kubectl delete pvc kafka-data-kafka-0 -n homework3**

Per forzare l'eliminazione dei dati si consiglia di utilizzare il seguente comando:

- **kubectl patch pvc kafka-data-kafka-0 -n homework3 -p '{"metadata":{"finalizers":null}}'**

6.2 Validazione Empirica tramite Query PromQL

Una volta avviato il sistema, la validazione del **White-Box Monitoring** avviene interrogando direttamente Prometheus.

Accedendo a localhost:9090, è stato possibile eseguire le seguenti query che dimostrano l'integrazione tra il codice applicativo e l'infrastruttura di monitoraggio.

1. Analisi del Carico Database (Throughput)

Questa metrica è fondamentale per dimensionare correttamente le risorse del database NoSQL.

- **Query:** `rate(mongo_total_query_total[5m])`
- **Analisi:** La query calcola la derivata temporale del contatore cumulativo, mostrandoci i "secondi di lavoro DB al secondo".

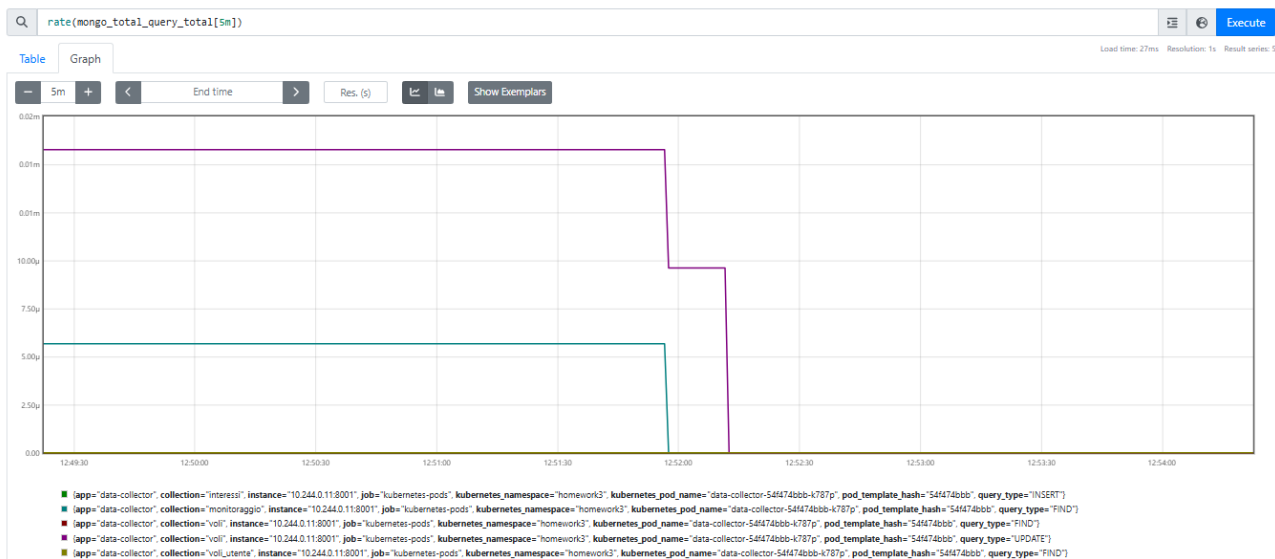


Figura 6.1 – Grafico del throughput su MongoDB. I picchi visibili corrispondono all'esecuzione dei job periodici di aggiornamento voli da parte del Data Collector, confermando il funzionamento ciclico del sistema.

2. Verifica della Resilienza (Circuit Breaker)

Questa è la prova empirica della capacità di "Self-Protection" del sistema. Abbiamo simulato un disservizio di rete verso l'API esterna per verificare l'attivazione delle protezioni.

- **Query:** `rate(opensky_requests_total{status="error"}[5m])`



Figura 6.2 – Rilevamento dell'apertura del Circuit Breaker. Il grafico mostra un valore positivo (> 0) esattamente nel momento in cui la connettività verso OpenSky degrada, dimostrando che il codice Python ha smesso proattivamente di inviare richieste per prevenire blocchi a cascata.

3. Debugging delle Performance (Latenza Puntuale)

Per analizzare la User Experience, monitoriamo la latenza puntuale delle operazioni critiche sullo User Manager (Login/Registrazione).

- **Query:** last_db_query_seconds

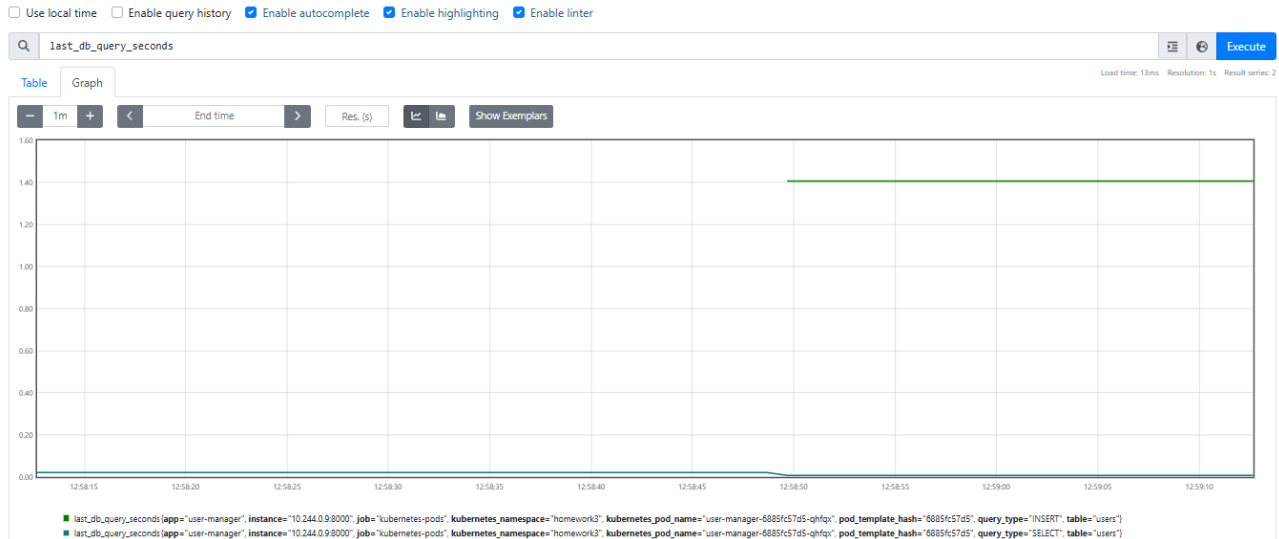


Figura 6.3 – Analisi della latenza delle query SQL. Le diverse linee colorate (distinte dalla label operation_type) permettono di identificare immediatamente se un rallentamento è dovuto a operazioni di lettura (SELECT) o scrittura (INSERT), facilitando il debugging mirato.

4. Conteggio Totale Notifiche (Cumulative Counter)

Questa metrica traccia il numero assoluto di email inviate correttamente dall'avvio del servizio fino al momento attuale. Serve a quantificare il lavoro totale svolto dal sistema.

Query: emails_total{status="success"}

Analisi: Il grafico mostra una linea crescente (monotona). L'altezza della linea indica il totale delle notifiche gestite; se la linea sale, significa che il sistema sta lavorando; se la linea è piatta, non ci sono nuovi invii in corso.



Figura 6.4 – Andamento del totale notifiche. La crescita costante della curva conferma che il servizio Notifier sta regolarmente processando i messaggi dalla coda e inviando le email di allerta.

5. Latenza di Integrazione SMTP (Performance Esterne) Poiché l'invio di email è un'operazione sincrona e bloccante verso un servizio esterno, è necessario monitorare i tempi di risposta per evitare che un rallentamento della rete blocchi il consumo dei messaggi da Kafka.

Query: email_latency

Analisi: Visualizza in tempo reale la durata esatta (in secondi) dell'ultima interazione completata con il server SMTP.

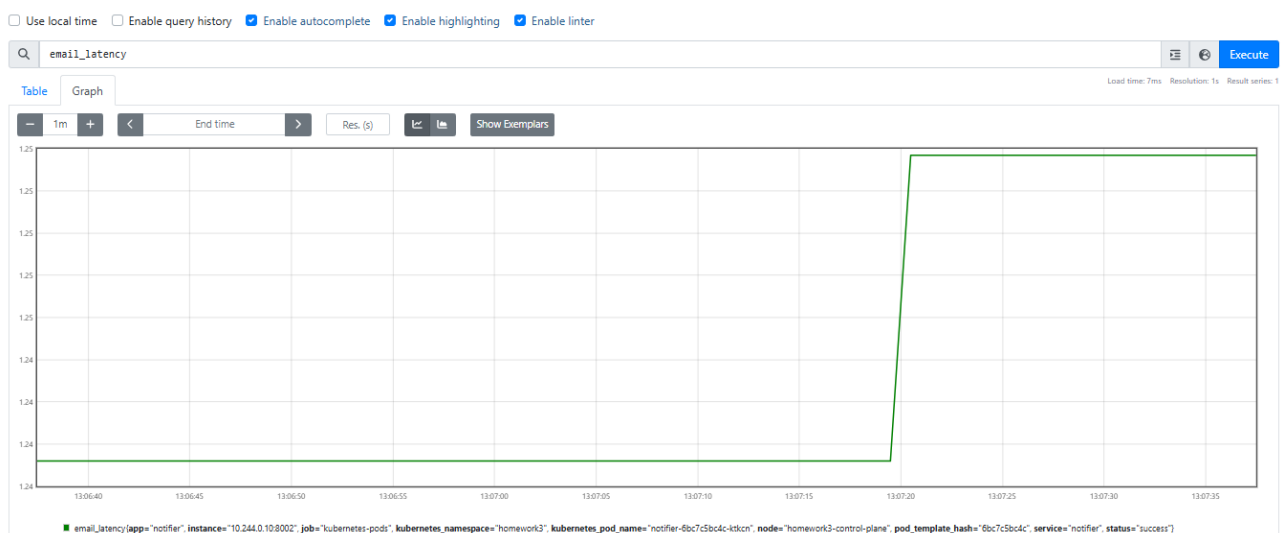


Figura 6.5 – Analisi della latenza SMTP. Il monitoraggio puntuale di questa metrica permette di distinguere tra rallentamenti interni dell'applicazione (es. elaborazione JSON lenta) e degradazioni delle performance della rete internet o del provider di posta elettronica.

7. CONCLUSIONI

Il presente progetto ha portato a compimento l'evoluzione del sistema di monitoraggio voli, trasformando un'applicazione a microservizi statica (Homework 2) in un'architettura **Cloud-Native** pienamente orchestrata e osservabile (Homework 3).

L'analisi finale del lavoro svolto evidenzia il raggiungimento di tre traguardi ingegneristici principali:

1. Gestione Dichiarativa dello Stato (StatefulSets & PVC)

Il passaggio da Docker Compose a Kubernetes non è stato un mero "porting" dei container, ma una riprogettazione della gestione dei dati.

Abbiamo superato la criticità della volatilità dei Pod adottando primitive native:

- **Identità Stabile:** L'uso di **StatefulSets** per *Kafka*, *MongoDB*, *PostgreSQL* e *Prometheus* ha garantito che i nodi critici mantengano un'identità di rete persistente (es. kafka-0), condizione necessaria per il corretto funzionamento di sistemi distribuiti che richiedono coordinamento.
- **Disaccoppiamento dello Storage:** Tramite i **Persistent Volume Claims (PVC)**, abbiamo separato il ciclo di vita del dato da quello del processo. I test di crash hanno dimostrato che il sistema è in grado di "guarire" (Self-Healing) riagganciando automaticamente i volumi ai nuovi pod, garantendo zero data loss.

2. Osservabilità White-Box "Deep-Dive"

L'integrazione di **Prometheus** ha segnato il passaggio da un monitoraggio passivo ("Il container è acceso?") a un monitoraggio attivo e semantico ("Come sta performando la query?"). Strumentando direttamente il codice Python (app.py) con la libreria *prometheus_client*, abbiamo reso visibili metriche interne altrimenti inaccessibili:

- La **Latenza delle Query DB** ci permette di ottimizzare le performance SQL/NoSQL.
- Il monitoraggio del **Circuit Breaker** (label status="circuit_open") offre una visione in tempo reale della stabilità delle dipendenze esterne (OpenSky API).

Grazie alla **Service Discovery** di Kubernetes, questa infrastruttura di monitoraggio è dinamica e scala automaticamente insieme all'applicazione, senza necessità di riconfigurazione manuale.

3. Ottimizzazione delle Risorse in Ambienti Vincolati

Operando in un cluster locale (Kind) con risorse limitate, abbiamo adottato un approccio rigoroso al **Capacity Planning**:

- **Retention Policy Calcolata:** La configurazione di Kafka (`log.retention.bytes=100MB`) non è un valore di default, ma il risultato di un calcolo ingegneristico per impedire la saturazione dei PVC da 250Mi, trasformando il broker in un buffer circolare robusto.
- **Quality of Service (QoS):** L'imposizione di Requests e Limits su CPU e RAM nel `deployments.yaml` protegge il cluster da fenomeni di "Noisy Neighbor" e memory leaks, garantendo stabilità anche sotto carico.

In conclusione, l'elaborato dimostra come la sinergia tra un'orchestrazione robusta (**Kubernetes**), una persistenza affidabile (**PVC**) e un'osservabilità profonda (**Prometheus**) permetta di realizzare sistemi distribuiti resilienti, manutenibili e pronti per la produzione.

4. Prospettive Future: Dal Cluster Locale al Cloud Pubblico (Scalabilità)

Sebbene l'attuale implementazione sia stata validata su un cluster a nodo singolo (Kind) per vincoli didattici, l'architettura è stata progettata per essere **agnostica rispetto all'infrastruttura**. L'uso di manifest standard CNCF pone le basi per una migrazione trasparente verso ambienti Cloud Enterprise. Gli sviluppi futuri prevedono:

- **Scalabilità Orizzontale Automatica (HPA):** Attualmente il sistema scala manualmente. In un ambiente Cloud (es. AWS EKS o Google GKE), implementeremmo l'**Horizontal Pod Autoscaler** (HPA) per aumentare automaticamente il numero di repliche dei microservizi *stateless* (User Manager, Data Collector) in base alle metriche di CPU o al throughput custom rilevato da Prometheus.

- **Adozione di Managed Services:** In un'ottica di produzione su larga scala, i componenti *stateful* (attualmente ospitati in container autogestiti) verrebbero migrati verso servizi gestiti dal provider Cloud (es. **Amazon RDS** per Postgres, **Confluent Cloud** per Kafka). Questo ridurrebbe l'onere operativo (backup, patching), permettendo al team di concentrarsi esclusivamente sulla business logic.
- **Gestione del Traffico Globale:** L'attuale Ingress Controller locale verrebbe sostituito da un **Cloud Load Balancer** per gestire traffico TLS su larga scala e distribuire il carico su diverse zone di disponibilità (Availability Zones), garantendo Disaster Recovery geografico.

In conclusione, l'elaborato dimostra come la sinergia tra un'orchestrazione robusta (Kubernetes), una persistenza affidabile (PVC) e un'osservabilità profonda (Prometheus) permetta di realizzare sistemi distribuiti che, pur nascendo in un ambiente di sviluppo locale, possiedono tutte le caratteristiche architetturali necessarie per scalare nel Cloud pubblico.